

Reviewer Pack — Minimal Tensor Rank of Hermitian Matrices Bounds BP Read-Twi...

Entry 0d7c1af18591 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 19:46:10 UTC

Minimal Tensor Rank of Hermitian Matrices Bounds BP Read-Twice Complexity

Entry ID: 0d7c1af18591 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 19:46:10 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Tensor Rank) **Field B** (complexity object): Complexity Theory: BP Read-Twice Complexity

Statement:

[‘For any read-twice branching program P , the minimal tensor rank of its associated Hermitian matrix is at most exponential in the depth of P .’, ‘Specifically, if P has depth d , then there exists a matrix M such that P can be approximated by a Hermitian matrix with minimal tensor rank $O(2^d)$.’, ‘For read-twice BP P_{IP_2} , which computes inner product over F_2 , the minimal tensor rank of its associated Hermitian matrix is at least $2^{n/4}$.’]

Rationale (proposer’s reasoning):

[‘The connection between quantum information theory and complexity theory through tensor rank has been explored in the context of matrix product states. However, the study of minimal tensor rank for read-twice BP complexity could reveal new insights into the structure of these programs.’, ‘Hermitian matrices naturally arise in the computation of quantum operations, which are closely related to branching programs. A precise relationship between tensor rank and read-twice BP complexity could lead to improved algorithms or lower bounds in the study of P vs NP .’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7861400ea390a448

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a read-twice BP of depth d and size $n \leq 40$, if the minimal tensor rank of its associated Hermitian matrix is $O(2^d)$ or greater than $2^{n/4}$ for BP IP_2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor rank AND Hermitian matrices AND BP read-twice complexity - Hermitian matrix minimal tensor rank AND quantum information theory AND BP read-twice - read-twice branching program AND inner product F₂ AND Hermitian matrix lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/2006.02374v2>] On tensor rank and commuting matrices - [<http://arxiv.org/abs/1912.07175v2>] Hermitian Tensor Decompositions - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/2311.01864v1>] SortNet: Learning To Rank By a Neural-Based Sorting Algorithm - [<http://arxiv.org/abs/2010.15321v2>] Coherent pre-order of quantum states - [<http://arxiv.org/abs/1902.02640v4>] Hermitian tensor and quantum mixed state - [<http://arxiv.org/abs/1308.2123v1>] The Control of Branching Morphogenesis - [<http://arxiv.org/abs/1312.5571v2>] Interim recommendations for the evaluation of Higgs production cross sections and branching ratios at the LHC in the Two

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        C = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        m = len(A)
        n = len(A[0])
```

```

augmented_matrix = [A[i] + [b[i]] for i in range(m)]

for i in range(n):
    max_row = i
    for j in range(i+1, m):
        if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
            max_row = j
    augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

    pivot = augmented_matrix[i][i]
    for j in range(i, n+1):
        augmented_matrix[i][j] /= pivot

    for j in range(m):
        if j != i:
            factor = augmented_matrix[j][i]
            for k in range(n+1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

x = [0 for _ in range(n)]
for i in range(n-1, -1, -1):
    x[i] = augmented_matrix[i][-1]
    for j in range(i+1, n):
        x[i] -= augmented_matrix[i][j] * x[j]
return x

def svd(A):
    m, n = len(A), len(A[0])
    U = [[0 for _ in range(m)] for _ in range(m)]
    S = [0 for _ in range(min(m, n))]
    Vt = [[0 for _ in range(n)] for _ in range(n)]

    A_t = list(zip(*A))

    # Compute U
    for i in range(m):
        u_i = [A[i][j] for j in range(n)]
        norm_u_i = math.sqrt(sum(x**2 for x in u_i))
        U[i] = [x / norm_u_i for x in u_i]

    # Compute Vt
    for j in range(n):
        v_j = [A_t[j][i] for i in range(m)]
        norm_v_j = math.sqrt(sum(x**2 for x in v_j))
        Vt[j] = [x / norm_v_j for x in v_j]

    # Compute S
    for i in range(min(m, n)):
        S[i] = sum(A[i][j]**2 for j in range(n))**(1/2)

    return U, S, Vt

def min_tensor_rank(n):
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    C = matrix_multiply(A, B)

```

```

    U, S, Vt = svd(C)
    return len([s for s in S if s != 0])

n = random.choice([5, 10, 15, 20, 30, 40])
depth = random.randint(1, 10)

instances_tested = 30
total_rank = 0

for _ in range(instances_tested):
    rank = min_tensor_rank(n)
    total_rank += rank

mean_rank = total_rank / instances_tested

conjecture_holds = mean_rank <= 2**depth or (n == 40 and mean_rank > n**2 / 4)

return {
    "metric_name": "min_tensor_rank",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean_rank={mean_rank}, depth={depth}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, "metric_name": "min_tensor_rank", "metric_value": 20.0, "instances_tested": 30, "conjecture_holds": true, "counterexample": ""}
TRIAL: {"seed": 23, "metric_name": "min_tensor_rank", "metric_value": 15.0, "instances_tested": 30, "conjecture_holds": true, "counterexample": ""}
TRIAL: {"seed": 37, "metric_name": "min_tensor_rank", "metric_value": 40.0, "instances_tested": 30, "conjecture_holds": true, "counterexample": ""}
TRIAL: {"seed": 53, "metric_name": "min_tensor_rank", "metric_value": 30.0, "instances_tested": 30, "conjecture_holds": true, "counterexample": ""}
TRIAL: {"seed": 71, "metric_name": "min_tensor_rank", "metric_value": 15.0, "instances_tested": 30, "conjecture_holds": true, "counterexample": ""}

```

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_048c6e36.py", line 121, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_048c6e36.py", line 101, in run_trial
    rank = min_tensor_rank(n)
            ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_048c6e36.py", line 91, in min_tensor_rank
    U, S, Vt = svd(C)
               ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_048c6e36.py", line 79, in svd
    Vt[j] = [x / norm_v_j for x in v_j]
              ~^~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed during execution, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12372
2	preregistration	ollama_remote	glm4:latest	0	0	5837
3	novelty	ollama_remote	glm4:latest	0	0	4733
4	novelty	ollama_remote	glm4:latest	0	0	5864
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15058
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13078
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13436
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15030
9	judge	ollama_remote	glm4:latest	0	0	25322

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110731 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d7c1af18591.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d7c1af18591.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0d7c1af18591.tar.gz` (if generated)